

□ **Suggerimento:** Collegamento Tematico

I concetti di ottimizzazione qui trattati si applicano anche al training delle reti neurali dove si usa Gradient Descent. Il teorema No Free Lunch vale per qualsiasi algoritmo di ottimizzazione, inclusi quelli del deep learning.

5.7.1 Ottimizzatori Globali

Gli algoritmi di ottimizzazione esaminati per la soluzione dei problemi di segmentazione e registrazione di immagini visti finora hanno la caratteristica di essere **ottimizzatori locali**. Questo implica che la soluzione trovata dipende fortemente dalle condizioni iniziali.

Sarebbe preferibile avere degli **ottimizzatori globali**, cioè algoritmi in grado di massimizzare una funzione indipendentemente dalle condizioni iniziali. Tuttavia, nella pratica gli ottimizzatori globali hanno prestazioni peggiori degli ottimizzatori locali, sia come capacità di trovare una soluzione migliore che come tempo di calcolo.

Ad esempio, un algoritmo a **ricerca esaustiva** (o *random search*) è sicuramente un ottimizzatore globale in quanto non ha condizioni iniziali, ma ha complessità computazionale molto alta.

5.7.1.1 Teorema “No Free Lunch”

Queste considerazioni si formalizzano nel cosiddetto **teorema “No Free Lunch”** (niente per niente).

□ **Citazione:** Enunciato

“Tutti gli algoritmi che ricercano il massimo di una funzione hanno esattamente la stessa prestazione media quando vengono provati su tutte le possibili funzioni.”

La figura illustra la differenza tra un algoritmo generico ed uno specializzato.

5.7.1.1.1 Formulazione Formale

Definiamo un algoritmo a come una funzione che partendo da un insieme di punti m nello spazio delle soluzioni produce un nuovo punto d con un certo valore y della funzione da ottimizzare f . Allora:

$$\sum_f P(d_m^y \mid f, m, a_1) = \sum_f P(d_m^y \mid f, m, a_2)$$

dove $P(d_m^y \mid f, m, a)$ è la probabilità di ottenere un punto nello spazio delle soluzioni con valore y , condizionato dalla funzione da ottimizzare f , dal numero di iterazioni m e dall'algoritmo usato a .

Il teorema è valido per ogni possibile coppia di algoritmi a_1 e a_2 , ed è stato dimostrato da **Wolpert e Macready nel 1997**.

5.7.1.1.2 Corollario

Per ogni possibile misura di prestazioni $M(d_m^y)$ (cioè ogni misura che ci dice quanto è “buona” la stima d_m^y per il problema che vogliamo risolvere), la media su tutte le f della $P(M(d_m^y) \mid f, m, a)$ è **indipendente da a** .

5.7.1.1.3 Implicazioni Pratiche

Il teorema afferma che per ogni misura di prestazioni, la media delle prestazioni su tutte le funzioni possibili è indipendente dall’algoritmo. Quindi:

- Se si **specializza** l’algoritmo su una particolare classe di funzioni → prestazioni **migliori** su quella classe
- Ma si otterranno prestazioni **peggiori** su tutte le funzioni non appartenenti alla classe selezionata

□ **Info:** Interpretazione Alternativa

Un algoritmo migliore della ricerca casuale su un set di funzioni incorpora informazioni (esplicite o implicite) sul problema da risolvere. Tuttavia, sull’insieme globale delle funzioni possibili, le prestazioni medie di qualsiasi algoritmo sono equivalenti a quelle di un algoritmo di ricerca casuale.

5.7.1.2 Approccio MultiStart

Un modo di realizzare un ottimizzatore globale è utilizzare un **ottimizzatore locale** variando le sue condizioni iniziali.

Dato un search-space S , supponiamo di avere un ottimizzatore locale in grado di convergere ad una soluzione locale A se le condizioni iniziali sono in uno spazio dS intorno ad A . Se dividiamo S in N parti tali che ogni singola parte sia più piccola di dS , possiamo sperare di trovare la soluzione ottima eseguendo N volte l’ottimizzatore locale.

□ **Esempio:** Esempio: Clustering

In un algoritmo di clustering possiamo ripetere il procedimento partendo ogni volta da condizioni diverse, come avviene attraverso l’opzione **replicate** del **kmeans** in MATLAB.

In generale questo approccio è detto **MultiStart** ed è implementato in MATLAB dalla funzione omonima.

5.7.1.2.1 Parallelizzazione

Definito il numero N di repliche dell’ottimizzatore locale, il tempo di calcolo (essendo i processi di ottimizzazione indipendenti) sarà mediamente N volte il tempo di una singola ottimizzazione.

Essendo i processi indipendenti, è possibile farli girare **in parallelo** su più processori (o core), ottenendo un tempo di calcolo N/C dove C è il numero di core.

La scelta delle condizioni iniziali può essere fatta:

- Dividendo in regioni il search-space
- Generandole in modo casuale

In generale, essendo il tempo di calcolo elevato, è fondamentale che l'algoritmo sia **parallelizzabile** (o *scalabile*), cioè in grado di girare in modo efficiente su una macchina parallela.

5.7.1.3 Legge di Amdahl

Idealmente, il tempo di calcolo di un algoritmo parallelizzabile dovrebbe scalare linearmente con il numero di processori. Di fatto lo **speedup** (cioè l'incremento di velocità ottenuto attraverso l'uso di una architettura parallela) sarà dato dalla **legge di Amdahl**:

$$S = \frac{1}{1 - p + \frac{p}{s}}$$

dove:

- p = percentuale del tempo di calcolo della parte parallelizzabile (calcolata sull'esecuzione sequenziale)
- s = speedup ottenuto facendo girare la parte parallelizzabile sulla macchina parallela
- S = speedup globale

5.7.1.3.1 Casi Limite

Condizione	Risultato
$p = 1$ (tutto parallelizzabile)	$S = s \rightarrow$ speedup globale = efficienza parallelizzazione
$p < 1$ e $s = N$ (caso ideale)	Per $N \rightarrow \infty$: $S = \frac{1}{1-p}$

□ **Attenzione:** Limite Fondamentale

Lo speedup è sempre **limitato dalla parte non parallelizzabile** dell'algoritmo.

Come si osserva in figura (tratta da Wikipedia, voce *Parallel Computing*), basta che una piccola parte dell'algoritmo non sia parallelizzabile per limitare in modo sostanziale lo speedup e rendere inutile l'uso di un numero elevato di processori.

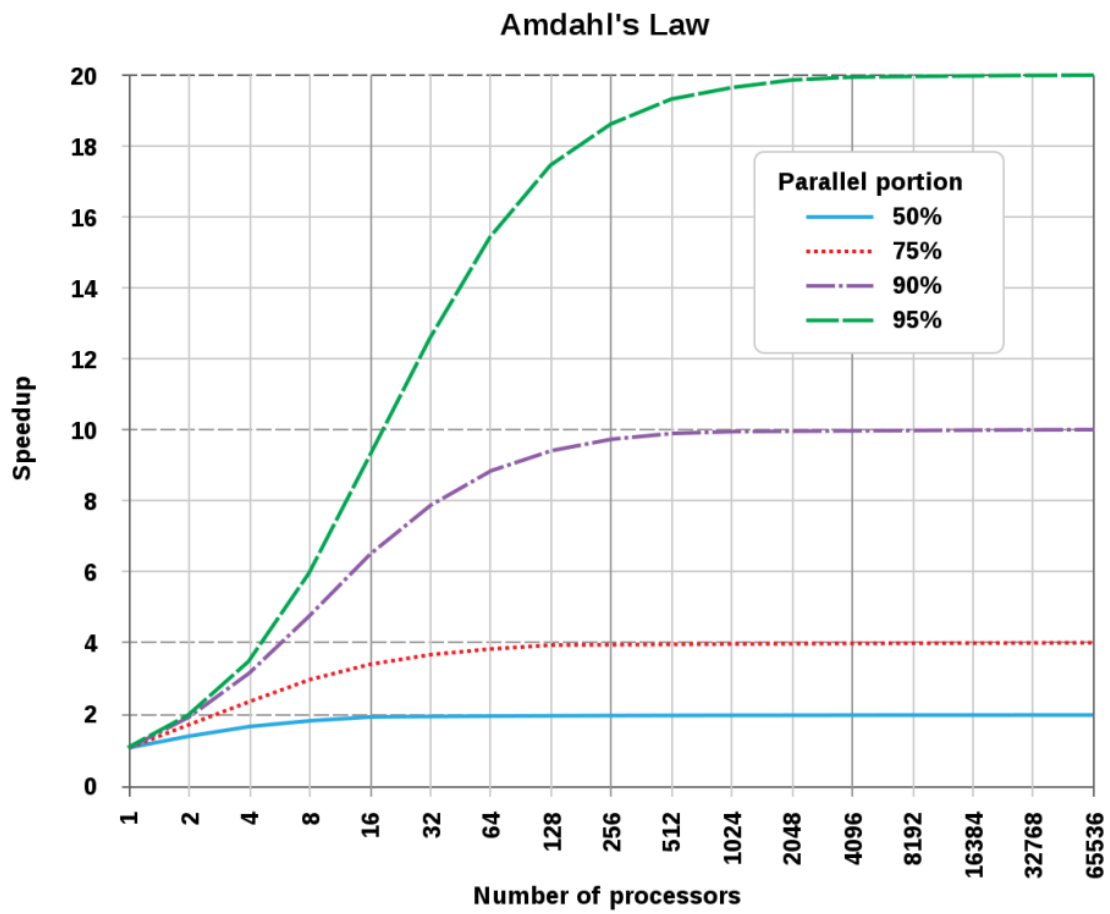


Figure 1: Pasted image 20251201190311.png

5.7.1.3.2 Componenti Non Parallelizzabili

Tipicamente la parte non parallelizzabile include:

- **Accesso ai dati** (da disco o memoria)
- **Raccolta dei risultati** dai processi paralleli
 - **Esempio:** Esempio: MultiStart

Nel MultiStart dobbiamo:

1. Leggere i dati e far partire i processi di ottimizzazione locale
2. Alla fine, ordinare i risultati secondo i valori ottenuti
3. Scegliere quello ottimo

5.7.1.3.3 Legge di Gustafson

Nella pratica, p dipende dalla quantità di dati: più dati devo analizzare, maggiore è il tempo che impiega la parte parallela rispetto a quella sequenziale.

- **Suggerimento:** Implicazione Pratica

Ha senso utilizzare un numero elevato di processori **solo per elaborare volumi di dati molto grandi** (legge di Gustafson).
